LLMs Know More About Numbers than They Can Say

Fengting Yuchi[1] **Li Du**[2] **Jason Eisner**[2]
[1]Shanghai Jiao Tong University and [2]Johns Hopkins University
yc1114@sjtu.edu.cn and {leodu,jason}@cs.jhu.edu

arXiv:2602.07812v2 [cs.CL] 17 Feb 2026

Abstract

Although state-of-the-art LLMs can solve math problems, we find that they make errors on numerical comparisons with mixed notation: "Which is larger, 5.7×10^2 or 580?" This raises a fundamental question: Do LLMs even know how big these numbers are? We probe the hidden states of several smaller open-source LLMs. A single linear projection of an appropriate hidden layer encodes the *log-magnitudes* of both kinds of numerals, allowing us to recover the numbers with relative error of about 2.3% (on restricted synthetic text) or 19.06% (on scientific papers). Furthermore, the hidden state after reading a *pair* of numerals encodes their *ranking*, with a linear classifier achieving over 90% accuracy. Yet surprisingly, when explicitly asked to rank the same pairs of numerals, these LLMs achieve only 50–70% accuracy, with worse performance for models whose probes are less effective. Finally, we show that incorporating the classifier probe's log-loss as an auxiliary objective during finetuning brings an additional 3.22% improvement in verbalized accuracy over base models, demonstrating that improving models' internal magnitude representations can enhance their numerical reasoning capabilities. Our code is available at github.com/VCY019/Numeracy-Probing.

1 Introduction

Large language models (LLMs) are increasingly used in mathematical (Jiang et al., 2023), scientific (Taylor et al., 2022; Zhang et al., 2025), financial (Wu et al., 2023), and engineering (Jimenez et al., 2024) domains. In these domains, numeracy—the ability to understand and reason about *numbers*—is an essential basic skill.

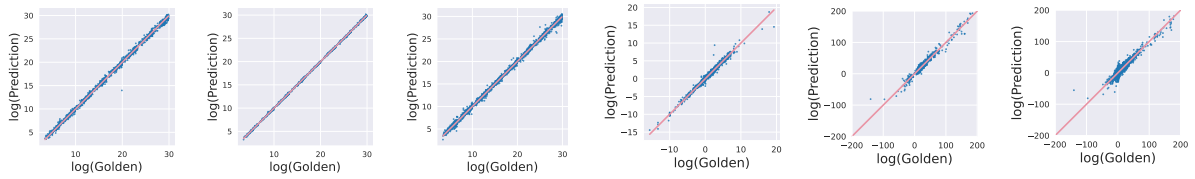
While LLMs can correctly answer questions such as "What is 5.7×10^2?" and "Which is larger, 570 or 580?", we find that even leading non-reasoning models like GPT-4.1 make many mistakes on cross-notation comparisons like "Which is larger, 5.7×10^2 or 580?" This discrepancy raises questions: Do LLMs know how big these numbers are, representing them internally in a way that supports intelligent discussion of scientific text? Or do they lack number sense and are specialized to specific manipulations of *numerals*—the textual representations of numbers—in specific notations?

Model	Verbalization Accuracy	Probing Accuracy
GPT-4.1	94.19	—
GPT-4.1-mini	92.12	—
DeepSeek-R1-Distill-Llama-8B	57.81	95.62
DeepSeek-R1-Distill-Qwen-7B	64.19	97.38
Llama-2-7b	50.81	98.44
Llama-3.1-8B-Instruct	55.06	94.81
Mistral-7B-v0.1	50.00	96.44
OLMo-2-1124-7B	53.44	93.50
Qwen3-8B	70.00	98.88

Table 1: Cross-notation comparison task on held-out data (see §B.1 for data details). **Verbalization accuracy** evaluates verbal responses using a one-shot prompt and no training, while **probing accuracy** measures performance of a linear classifier trained on hidden states. All values are percentages without the % sign. Despite hidden states containing rather accurate comparison information extractable via linear probes, verbalization performs substantially worse.

To study these questions, we first probe the internal numerical knowledge of smaller LLMs (7B–8B parameters) by training linear probes on hidden states of numeral tokens, following Zhu et al. (2025). We train and test on two kinds of text: synthetic prompts and real-world arXiv papers. We find that standard linear regression can successfully predict log-magnitudes (§3.1). For example, Mistral-7B has layers that linearly encode numerals to ~2.3% median relative error on synthetic data and ~19.06% on scientific papers.

Second, we show that these smaller LLMs also internally *compare* numbers (§3.2). A linear binary classifier (trained with logistic regression) reveals which of the two numbers in the question is bigger, even when the numbers are so close that their predicted magnitudes (using the previous method) are too noisy for reliable comparison.



(a) Synth, dec (18) (b) Synth, sci (31) (c) Synth, mixed (30) (d) ArXiv, dec (5) (e) ArXiv, sci (5) (f) ArXiv, mixed (10)

Figure 1: Scatterplots of predicted vs. true ("golden") log-magnitudes for Mistral-7B (32 layers) across different datasets and notation types. "Synth" refers to the synthetic cross-notation data we constructed in §2. Dec/sci/mixed conditions train and test on decimals only, scientific notation numerals only, and both, respectively. We train and test a probe at each layer, and then plot performance on held-out test data for only the probe that achieved the highest R^2 on held-out validation data. The parenthesized number is the layer index of that probe.

Despite having done these computations internally, these LLMs struggle significantly at *verbalizing* the cross-notation comparison (§3.3). When prompted to say which of two cross-notation numerals is larger, the models achieve only 50–70% accuracy—far closer to the 50% random baseline than their internal knowledge would predict. This sharp performance gap indicates that LLMs possess numerical knowledge that they cannot reliably verbalize through language generation.

Still, probing effectiveness in early layers does *correlate* well with verbalized performance (§3.4). For a probe of either type (applied to the first 3 layers), probe accuracy predicts the accuracy of verbal output (produced by the last layer) across diverse 7B–8B LLMs (Figs. 5, 6 and 9).

To test whether this connection is causal, we experiment with adding the classifier probe's training loss to the LLM's log-loss training objective on the task of verbalized cross-notation comparison (§3.5). Most LLMs do achieve higher verbalized comparison accuracy when fine-tuned with this augmented objective rather than the standard objective (Table 2). This finding suggests that targeting internal representations is a promising approach to improving numeracy in LLMs. That is, models know more than they can say—but improving what they know improves what they can learn to say.

To summarize our key findings:

- Numerical log-magnitudes and magnitude comparisons are encoded linearly, for various LLMs, datasets, and numeral notations.
- Even so, LLMs verbalize cross-notation comparisons poorly.
- Tuning a model's internal representations to better encode numeric comparisons helps the model better answer such comparison questions verbally.

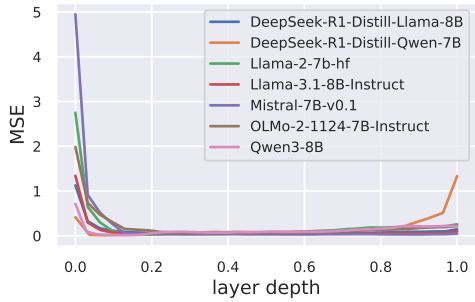


Figure 2: MSEs in log-space of regression probes on cross-notation data of each LLM across layers.

2 Experimental Setup

Data We construct a synthetic `cross-notation` dataset to support our studies of both regression and comparison. To study regression probing on realistic scientific text, we derive an `arXiv` dataset from the peS2o dataset (Soldaini and Lo, 2023). See §B for details of both datasets.

In `cross-notation`, numeral pairs are constructed so that each pair consists of one numeral in scientific notation and the other in normal form (e.g., 5.7×10^2 vs. 560).[1]

For verbalization on `cross-notation`, we prompt with a one-shot demonstration since the model has not been trained for the task (see §C for more details):

```
Q: Which is larger, 9.9 × 10^2 or 100? A: 9.9 × 10^2
Q: Which is larger, {a} or {b}? A:
```

In contrast, we train and test our `cross-notation` probes on a zero-shot prompt (no demonstration), to better isolate the model's intrinsic encoding:

```
Q: Which is larger, {a} or {b}? A:
```

[1]We found LLMs can compare numerals of the same notation with 100% accuracy.

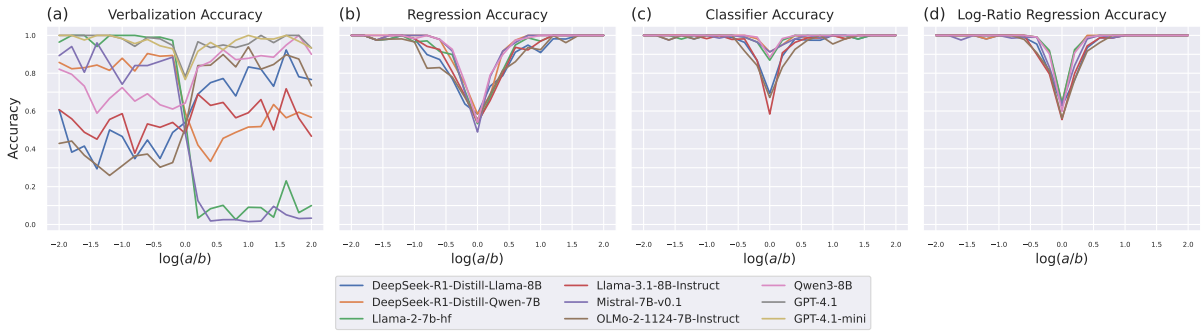


Figure 3: Accuracy of cross-notation comparison ($a \overset{?}{>} b$) versus the relative magnitude of the two numbers ($\log_2(a/b)$). (a) Verbalized comparison using one-shot prompting; (b) Comparison of the two values predicted by regression; (c) Comparison via a classification probe; (d) Comparison via the log-ratio predicted by regression. Note that (b)–(d) require access to the hidden states, so they do not include the large closed-source models GPT-4.1 and GPT-4.1-mini. See Fig. 10 for individual models' results.

Probing For each layer of the transformer LLM, we extract hidden states from the zero-shot prompt and train separate probes. Linear regression probes are trained on the last tokens of the numerals to predict their $\log_2$ magnitudes and the last token of the prompt to predict $\log_2(a/b)$. Linear classification probes (using logistic regression) are trained on the last token of the prompt to predict whether `{a}` $>$ `{b}`. See §D for details.

Finetuning To investigate whether internal magnitude representations can be leveraged to improve numerical reasoning, we incorporate auxiliary probing loss during finetuning on `cross-notation`. We add probe heads to a single layer and finetune on the standard language modeling loss *plus* the classification probing loss,[2] $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{LM}} + \beta\mathcal{L}_{\text{cls}}$, where the hyperparameter β controls the strength of auxiliary supervision. See §E for details.

Models To ensure that the experimental results are not specific to a certain model's architecture or training procedure, we select a set of widely used open-weight 7B–8B LLMs, including base pretrained, instruction-tuned, and distilled models. We additionally evaluate GPT-4.1 and GPT-4.1-mini (OpenAI's non-reasoning frontier models) to assess whether verbalization difficulties persist in larger, more capable models.

3 Experimental Results

3.1 Magnitude Probing

To investigate whether LLMs internally encode numerical magnitude information, we first train linear regression probes on hidden states to predict the $\log_2$ values of numerals. Fig. 1 shows scatterplots of predicted versus true log-magnitudes for Mistral-7B across different datasets and notation types.

The strong correlation observed in Fig. 1 suggests that, on the `cross-notation` dataset, log-magnitudes of decimals and scientific notation numerals are encoded linearly in the LLM's hidden representations. Even in mixed probing, where probes are trained and tested on both decimal and scientific notation tokens, linear regressors achieve high performance on `cross-notation` with $R^2 > 0.998$ (Fig. 1c).

On `arXiv` (Fig. 1f), the linear probe is not as precise, though it still achieves a correlation of $\rho = 71\%$ ($R^2 = 0.51$). Numbers in `arXiv` play many more different *semantic roles* (Lo et al., 2020, App. F). Perhaps these are represented in different subspaces, so that a single linear probe is not powerful enough.

Even so, this latter model can predict magnitudes on held-out `arXiv` data to 19.06% median relative error. Thus, this layer of Mistral-7B contains a noisy invariant linear representation of each numeral's magnitude—which the next layer's attention mechanism could learn to extract through linear query/key/value projections, providing "number sense". Note that the better 2.3% median relative error on `cross-notation` (Fig. 1c) is possible only because the models are overfitted to very specific prompt formats.[3]

[2] We experimented with adding regression probing losses but found no improvement in performance (see §E).

[3] These models are also underdetermined because they have only seen one kind of data. They do not generalize well even within the `cross-notation` dataset: the Fig. 1a model performs much worse on the Fig. 1b data ($R^2 = 0.56$), and vice-versa ($R^2 = 0.34$).